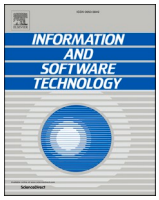
Information and Software Technology 196 (2026) 108149



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof





Allowing test execution to continue after failures: tool support and an industrial case study for fault-tolerant online model-based testing

Vəhid Gəruslu [a,b,1,*], Alper Buğra Keleş [c], Yunus Balaman [d], Glen O'Donovan [e]

[a] Queen's University Belfast, Belfast, United Kingdom
[b] Azerbaijan Technical University, Baku, Azerbaijan
[c] Testinium A.Ş., Istanbul, Türkiye
[d] KG Teknoloji A.Ş., Istanbul, Türkiye
[e] Lunatech, Rotterdam, The Netherlands

ARTICLE INFO

Keywords:
Model-based testing (mbt)
Fault tolerance
Test execution resilience
Graphwalker
Flaky tests
Exception handling
Software testing tools
Industrial case study

ABSTRACT

Context: Model-Based Testing (MBT) can be executed in either offline or online mode. While offline MBT pre-generates test cases that can be independently executed and resumed, online MBT dynamically interacts with the system under test by generating test steps on-the-fly based on runtime feedback. This adaptive exploration makes online MBT particularly attractive in industrial contexts; however, it also introduces a structural fragility: when an unexpected failure occurs during execution, the testing session often terminates prematurely, preventing further exploration of the model.

Objective: The objective of this study is to improve the robustness and usability of online MBT by enabling execution to continue in the presence of localized assertion failures without masking genuine faults.

Method: We propose two lightweight execution-level fault-tolerance heuristics: Retry Failed Node and Blacklist Failed Node. The heuristics were implemented in the GraphWalker MBT tool by extending its execution and exception-handling mechanisms and evaluated using a large-scale industrial MBT suite at Testinium.

Results: The retry heuristic mitigates transient and flaky failures, while the blacklist heuristic enables continued traversal after persistent localized failures. Together, they significantly reduce premature termination of online MBT execution with minimal and bounded overhead.

Conclusion: Fault-tolerant execution semantics are essential for making online MBT practical in real-world testing. The proposed heuristics improve execution continuity and support broader industrial adoption of online MBT.

1. Introduction

Model-Based Testing (MBT) is a black-box testing approach that enables automated test generation (test-case design) from certain types of behavioral models [1,2].

Although MBT has existed for over 55 years now—starting with a technical report [3] by IBM in 1970 presenting a MBT test tool to generate test cases based on Cause-Effect Graphs "*most developers [still] don't view MBT as a mainstream [testing] approach*" [4]. Numerous studies have mentioned key barriers to industrial MBT adoption [2,4–9], including: (1) lack of awareness and training about MBT, (2) issues with and limitations of existing MBT tools, and (3) non-trivial upfront modeling effort.

Referring to the item #2 above, many existing MBT tools have one key limitation: test execution is immediately terminated upon encountering an assertion failure. In other words, once a single fault is detected by the test model in execution, the remaining parts of the model are not exercised, and test generation halts prematurely. This "strict" failure-handling behavior reduces the practicality and robustness of MBT in large-scale or long-running test scenarios. The afore-mentioned failure-handling behavior is also often referred to as "fail-fast" [10–12].

In real-world industrial testing, such strict fail-fast behavior can be counterproductive. It leads to test session interruptions due to *flaky tests* [13] —tests that intermittently fail due to non-deterministic issues such

* Corresponding author.
E-mail addresses: v.garousi@qub.ac.uk, vahid.garousi@aztu.edu.az (V. Gəruslu), alper.keles@testinium.com (A.B. Keleş), yunus.balaman@kgteknoloji.com (Y. Balaman), glen.odonovan@lunatech.nl (G. O'Donovan).
[1] Alternate name spelling: Vahid Garousi.

https://doi.org/10.1016/j.infsof.2026.108149
Received 13 January 2026; Received in revised form 11 March 2026; Accepted 10 April 2026
Available online 12 April 2026
0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

as timing delays or temporary UI inconsistencies. These minor, often non-critical assertion failures—such as checking whether a UI element is visible—could be logged and skipped without stopping the remainder of the test session. However, without built-in fault-tolerance mechanisms, current MBT tools (all those that we have assessed [14]) make inefficient use of execution effort and limit model coverage whenever such a non-critical failure is encountered.

This paper addresses the above limitation by proposing and evaluating two lightweight *fault-tolerance heuristics* for GraphWalker. The first heuristic retries a failed node using a configurable bounded retry mechanism, allowing recovery from transient issues [15]. The second heuristic skips over the failed node, blacklists it, and continues executing the rest of the model from the last valid state. Both heuristics were designed with minimal impact on model logic and are easily integrated into GraphWalker's exception-handling mechanism.

The contributions of this paper are as follows:

1. We identify and formalize the limitations of fail-fast execution in MBT tools using real-world testing scenarios
2. We design and implement two lightweight fault-tolerance heuristics in a widely used MBT tool (GraphWalker)
3. We empirically evaluate the effectiveness of these heuristics using MBT models developed for a commercial test management system (Testinium) in an industrial context
4. We provide insights and lessons learned for practitioners seeking to improve MBT fault-tolerance in their test automation projects

In this paper, we focus on improving the fault-tolerance of MBT executions—i.e., enabling test sessions to continue in the presence of minor failures. This contributes to overall MBT resilience, but we use fault-tolerance as the main technical term throughout the paper.

The remainder of this paper is structured as follows. Section 2 provides background on MBT and the GraphWalker tool, as well as related work on failure handling in test automation. Section 3 describes the design of the two proposed fault-tolerance heuristics. Section 4 details their implementation within GraphWalker. Section 5 presents the empirical evaluation conducted using real-world MBT models from an industrial system under test. Section 6 discusses key insights, trade-offs, and applicability to other tools. Section 7 concludes the paper and outlines directions for future work.

2. Background, motivations, and related work

Model-Based Testing (MBT) has matured over several decades and has been successfully applied in a wide range of academic and industrial contexts. Despite this maturity, practical adoption of MBT continues to face notable challenges, particularly when MBT is applied at scale in real-world automated testing environments. One such challenge concerns the handling of execution-time failures during MBT test runs. This section provides the necessary background to understand this problem by first motivating the need for fault tolerance in automated test execution, then reviewing existing academic and industrial work on failure handling, and finally analyzing why these issues become especially critical in the context of online MBT. The discussion establishes the foundation for the fault-tolerance heuristics proposed in this paper.

2.1. Need for fault tolerance in automated test execution

Automated test execution plays a central role in modern software development by enabling frequent and repeatable validation of software behavior. Most test automation tools, however, rely on a fail-fast strategy, in which the occurrence of an assertion failure immediately terminates the ongoing test run. While effective for rapid fault detection, this strategy implicitly assumes that all observed failures are persistent and globally significant.

In practice, automated test execution is exposed to a range of execution-time conditions—such as timing variability, asynchronous system behavior, environmental instability, and tool-level brittleness—that can lead to non-persistent failures, commonly referred to as test flakiness. Under fail-fast execution semantics, such flaky failures can prematurely stop testing, even when the failure is localized and does not invalidate the remainder of the test execution.

This limitation becomes particularly problematic in long-running or exploratory test executions, where the objective is not only to detect failures but also to systematically explore system behavior and assess coverage across a broad execution space. Premature termination in these settings reduces coverage and limits the diagnostic value of test results.

Fault tolerance in automated test execution addresses this problem by enabling test runs to continue in the presence of certain execution-time failures, while still recording and reporting those failures. Rather than suppressing failures, fault-tolerant mechanisms aim to distinguish between failures that are transient or localized—such as flaky failures—and those that warrant immediate termination. By doing so, fault tolerance improves the robustness and effectiveness of automated testing, providing richer feedback and higher coverage without masking genuine defects.

A review of existing academic and industrial work indicates that fault tolerance in automated test execution is realized through a number of mechanisms. One such mechanism is limited re-execution of a failed test case, where a failing test or test step is retried a bounded number of times to distinguish transient failures [15] from persistent ones. Another mechanism focuses on failure isolation, whereby test cases that repeatedly fail are temporarily excluded or quarantined so that the remainder of the test execution can proceed. We provide a more detailed review of both academic and grey literature on this topic in the next subsection.

2.2. Existing work on fault-tolerant test execution

Building on the motivation above, understanding how fault tolerance is addressed in automated test execution requires considering evidence from both academic research and industrial practice. Related guidelines and methodological studies [16–18] have shown that relying solely on peer-reviewed literature can provide an incomplete view of software engineering topics that are strongly driven by tools and practitioner experience. In particular, many practically relevant techniques are reported primarily in grey literature, such as industrial blogs and tool documentation, and may not appear in academic venues. Following these recommendations, we provide next a brief review of existing work on fault-tolerant test execution by synthesizing findings from both academic and industrial sources.

2.2.1. Academic literature

Academic research has long recognized that automated test execution is vulnerable to execution-time failures that may not correspond to persistent faults in the system under test, yet can prematurely terminate testing [19,20]. Many testing tools adopt a fail-fast strategy, whereby the first assertion failure halts execution. While effective for rapid fault detection, this strategy reduces test completeness when failures are localized or transient.

Empirical studies on flaky tests provide strong evidence that re-executing identical test logic can yield different outcomes without any change to the system under test. Luo et al. showed, in a large-scale empirical study at Google, that a substantial proportion of observed test failures were non-deterministic and disappeared upon re-execution, motivating the use of bounded retries to distinguish persistent from transient failures [19]. Bell et al. further demonstrated that such non-deterministic failures significantly degrade test reliability and CI effectiveness, reinforcing the need for execution-time mitigation mechanisms beyond immediate termination [20].

Those category of approaches align with our first heuristic: retry a failed node in MBT execution (Section 3.1). Rather than treating a single assertion failure as definitive, allowing a limited reattempt enables the

test engine to determine whether the failure persists.

A complementary body of research has examined failure containment and isolation in test execution. Studies on test order dependency and shared state effects show that failures often affect individual test units, not entire test suites, and that isolating failing units prevents cascading failures across executions [20]. Although typically framed at the test-case or test-suite level, this idea conceptually aligns with our second heuristic: blacklist failed node (Section 3.2), where persistently failing elements are isolated so that the remaining test space can still be explored.

However, existing academic work overwhelmingly treats retries and isolation as external execution controls applied to linear test artifacts. The authors are not aware of any prior work on treatment of model-level fault tolerance during online model traversal in MBT, where execution decisions must be made dynamically as the model is explored in the MBT test process.

2.2.2. Industrial grey literature

Industrial practice strongly reinforces the relevance of retry and isolation mechanisms for improving the robustness of automated test execution [21]. Most modern test automation frameworks and CI pipelines support automatic retries of failed tests or test steps. This practice directly corresponds to our retry failed node heuristic, albeit applied at the level of test cases rather than model elements.

For example, the Maven Surefire plugin allows failed tests to be rerun a configurable number of times before declaring failure.[3] Similarly, a JUnit 5 extension pack named *JUnit Pioneer* (junit-pioneer.org) supports bounded retries, which allows developers to annotate tests with retry semantics such as @RetryingTest(n) or conditional retry policies. Examples of those two features are shown in Fig. 1. In the left-hand-side example, the test is automatically re-executed up to three times if it fails, allowing transient execution-time failures (e.g., temporary service unavailability) to be tolerated before the test is considered persistently failing. In the second JUnit test-code example, the test is retried only when it fails due to assertion errors, enabling selective fault tolerance while avoiding retries for other failure types (e.g., configuration or infrastructure errors).

These APIs embody a common industry pattern: re-execute a failing test a bounded number of times, and only treat the failure as persistent if all attempts fail.

In addition to retries, industry widely employs isolation mechanisms for persistently failing tests. Jenkins provides plugins such as Flaky Test Handler and Quarantine, which allow repeatedly failing tests to be marked or excluded so that the remainder of the test suite can continue executing [22]. Large-scale industrial reports from Google and Microsoft describe internal infrastructures that detect repeated failures and suppress or quarantine them to avoid blocking CI pipelines [23]. Slack and Atlassian report similar mechanisms, where failing tests are automatically excluded from critical pipelines while execution continues on unaffected tests [23,24].

These practices correspond closely to our blacklist failed node heuristic, in which a persistently failing model element is dynamically excluded from further traversal. Importantly, industrial solutions typically implement this logic outside the test generation process, at the level of CI orchestration or test runners. Consequently, isolation decisions lack awareness of model structure, traversal strategy, or coverage implications.

2.2.3. Positioning of the proposed approach w.r.t. related work and approaches

Existing approaches to handling failures in automated testing generally fall into three broad categories. First, many test automation frameworks adopt a strict fail-fast execution strategy, where test execution stops immediately after the first assertion failure. This approach prioritizes early fault detection but can significantly reduce coverage during long-running automated test sessions. Second, some test frameworks support test-level retry mechanisms, where an entire test case is re-executed after a failure in order to mitigate flaky test behavior. While this approach can recover from transient failures, it typically operates at the level of individual test cases rather than at the level of model traversal. Third, certain testing environments allow manual exclusion or skipping of failing tests, but such mechanisms are usually configured statically and do not dynamically adapt execution paths during runtime.

In contrast, the heuristics proposed in this paper operate directly at the execution-control layer of an online MBT engine. Rather than retrying entire tests or statically disabling failing scenarios, the proposed mechanisms dynamically adapt model traversal decisions at runtime. The retry heuristic allows recovery from transient failures through bounded node-level retries, while the blacklist heuristic enables continued exploration of the model by temporarily excluding persistently failing nodes. This execution-level adaptation is particularly suited to online MBT, where traversal decisions are made dynamically during execution and failures at individual model nodes can otherwise terminate the entire test session.

2.3. Motivations and need analysis

This subsection analyzes why strict fail-fast execution semantics pose practical challenges for online Model-Based Testing (MBT), particularly in industrial settings. We examine the characteristics of online MBT execution, distinguish it from offline MBT, and review common failure-handling strategies in existing MBT tools. Drawing on industrial experience, we show that fail-fast behavior can prematurely terminate MBT executions and limit their practicality, motivating the need for the fault-tolerance heuristics proposed in this paper.

2.3.1. MBT execution environment and toolset in the study context

This section does not aim to provide a general overview of Model-Based Testing (MBT). Instead, it describes the specific execution context in which MBT was applied in our study. The focus is on the runtime characteristics of the selected tool and its failure-handling behavior, as these aspects directly influence the fault-tolerance challenges addressed in this work.

In our study, MBT was applied in an online execution mode, where the model is traversed dynamically at runtime and test steps are generated and executed sequentially. This stateful execution model means that failures can interrupt the traversal process and prematurely terminate exploration, making failure-handling strategies particularly relevant.

To support online MBT execution in this study, we have selected *GraphWalker*, an open-source tool that dynamically traverses graph-based models at runtime (graphwalker.github.io). Through a systematic tool evaluation process [14], we adopted this tool for our MBT practices in year 2019, and have used it since then in our industrial setting [14]. We thus provide a brief overview of the tool, since the reverse engineering approach and tool reported in this paper is in the context of this MBT tool.

GraphWalker uses finite-state machine (FSM)-like models, composed of vertices (states/pages) and edges (transitions/actions). Test cases are generated dynamically based on traversal algorithms such as random walk, edge-coverage, or A* pathfinding. GraphWalker has been used in both academic and industrial settings for MBT of web and mobile applications [25–27].

[2] github.com/GraphWalker/graphwalker-project/blob/master/graphwalker-core/src/main/java/org/graphwalker/core/machine/FailFastStrategy.java

[3] maven.apache.org/surefire/maven-surefire-plugin/examples/rerun-failing-tests.html

```
@RetryingTest(3)
void shouldEventuallyPass() {
    assertTrue(externalService.isAvailable());
}
```

```
@RetryingTest(3)
@RetryOnFailure(AssertionError.class)
void retryOnlyOnAssertionFailure() {
    assertEquals(expected, actual());
}
```

Fig. 1. Examples of the bounded test-case retry feature in JUnit 5.

GraphWalker also integrates with Selenium-based test automation and supports embedding test logic directly into edge or vertex annotations. Its flexibility and support for JSON-format models has made it the ideal tool for our approach and context, as reported in [14]. Therefore, MBTModelGenerator is designed to produce output model files, fully compatible with GraphWalker's model execution engine, allowing immediate use for automated test runs.

It would be informative for the reader at this point to see a concrete example of MBT test artifact in the GraphWalker tool. One of the large MBT projects that we reported in [14] was the MBT project who goal was to test one of the company's own tools (developed in-house). The tool is also called by the company name, Testinium (testinium.io), which is a web-based test management tool, based on the Selenium framework. We show in Fig. 2 the Login page and the main Dashboard page of this example SUT. Upon a successful login by the user, the system would show the Dashboard page.

A MBT best-practice when using the GraphWalker tool is to follow modular design, and to design a separate MBT model for each web page of a given SUT [2,14]. We show in Fig. 3 one of the 18 MBT test models of a large MBT test suite for testing the *login* page [14]. Names of the other 17 test models of this SUT can be seen in the "tabs" in top of the web-based test tool view in Fig. 3, and they can be accessed from those tabs.

This *Login* test model verifies the various valid and invalid login scenarios supported by Testinium, including: (1) Standard login with correct and incorrect credentials, (2) Third-party logins (Google, LinkedIn, GitHub), (3) Forgot password flows for registered and unregistered users, and (4) Input validation (e.g., empty fields, invalid email formats).

Each state (node), labeled as v_*, represents a user-visible application screen or condition, e.g., (1) v_Start: The starting point of the test sequence, and (2) v_Verify_In_Login_Page: Main login interface with multiple options. Test assertions / verifications are developed for each node (more on that aspect below). Transitions (edges) are labeled as e_*, and each transition is a user action (e.g., clicking a button), e.g., e_Click_Signin.

In the context of the GraphWalker tool, test engineers need to provide the behavior of each node and edge of the MBT model, in Java using the Selenium framework. For example, for the edge e_valid_login in Fig. 3, we developed the Java test code shown in Fig. 4. In this example Selenium Java code, to conduct a valid login, the username and password fields are first located. Then, a correct combination of username and password values are entered in those fields. To find the HTML button for the "Sign in", a CSS selector path is given. The sign-in button is finally clicked programmatically.

More details about the MBT practice in GraphWalker and our MBT projects have been reported in [14]. Furthermore, several extensive demo videos can be found online [28].

2.3.2. Two types of model-based testing (MBT): offline and online

MBT can be broadly categorized into two execution modes: offline MBT and online MBT, distinguished by when test cases are generated relative to their execution [29,30]. Widely used MBT tools such as GraphWalker support both execution modes.

In offline MBT, the test model is traversed upfront to generate a finite set of concrete test cases prior to execution. These test cases are then executed independently by a test automation framework. In this mode, model traversal is completed before execution begins, and execution failures do not influence the exploration of the model. If certain generated test cases fail during execution, they are simply reported as failed, while the remaining test cases continue to run. Consequently, there is no risk of prematurely terminating model exploration due to execution-time failures, and model-level fault tolerance is not required.

In online MBT, by contrast, model traversal and test execution are tightly coupled. The MBT engine dynamically decides which model element (state or transition) to execute next while interacting with the system under test at runtime, guided by traversal strategies and coverage goals. Execution progress and model exploration therefore evolve together during a test run.

Online MBT is often preferred in practice, particularly for large or evolving systems, because it avoids the need to pre-generate and manage large test suites, supports adaptive exploration of the model, and enables long-running or exploratory testing guided by runtime coverage feedback. However, this tight coupling also introduces a key challenge: execution-time failures directly affect model traversal. As observed during our systematic assessments conducted in earlier phases of our R&D project [14], the across more than ten major MBT tools supporting online execution — including GraphWalker — the default behavior upon assertion failure was to terminate execution immediately, thereby preventing further exploration of the MBT model..

As a result, fault tolerance becomes a concern specifically in online MBT execution, where localized execution failures can prematurely stop model traversal and reduce coverage. In offline MBT, such mechanisms are unnecessary at the model level, since failures do not interfere with model exploration. This distinction motivates the focus of this paper, which addresses fault tolerance exclusively in the context of online MBT.

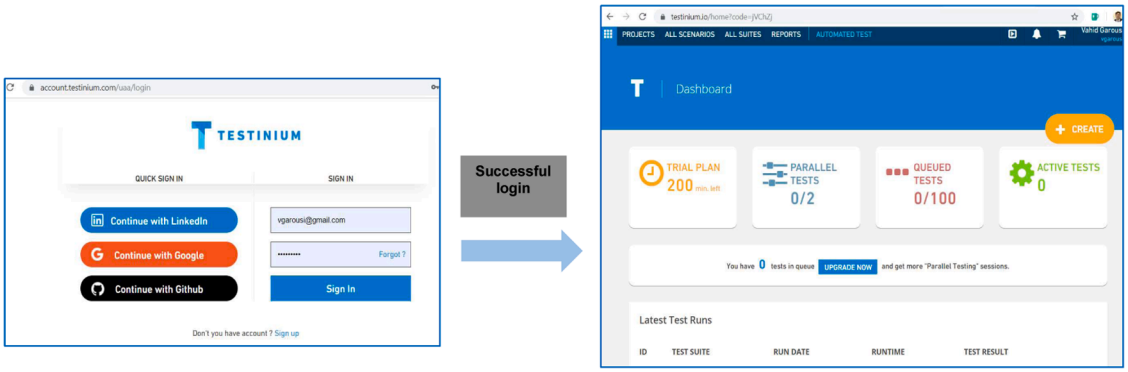


Fig. 2. Login feature of the example SUT: Testinium (testinium.io) [14].

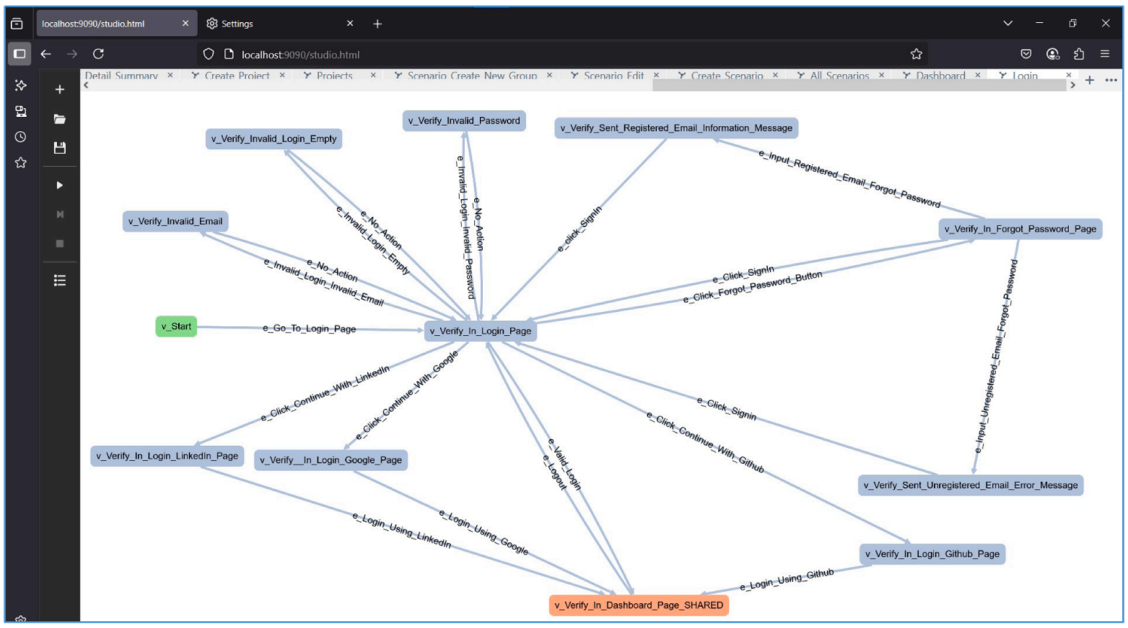


Fig. 3. One of the 18 MBT test models for testing the *login* page of the SUT: Testinium (testinium.io), opened in the test tool: *GraphWalker Studio*. Names of the other 17 test models can be seen in the "tabs" in top of the web-based test tool, and they can be accessed from there.

```
public void e_valid_login() {
   WebElement userNameElement =
        methodsPage.findElement(By.id("username"));
   userNameElement.clear();
   userNameElement.sendKeys(email);
   WebElement passwordElement =
        methodsPage.findElement(By.id("password"));
   passwordElement.clear();
   passwordElement.sendKeys(password);
   methodsPage.findEleminrent(By.cssSelector(
     "input[class$=\"login-page__submit-btn\"][value=\"Sign In\"]")).click();
}
```

Fig. 4. Java Selenium code implementing the behavior for edge e_valid_login in Fig. 3.

2.3.3. *Common test-failure handling strategies in MBT tools (e.g., fail-fast)*

In online Model-Based Testing (MBT), test execution is tightly coupled with model traversal, meaning that the system under test is exercised dynamically as the MBT engine explores states and transitions. Consequently, execution-time failure handling policies strongly influence how much of the model can be exercised during a test run.

Most existing MBT tools adopt a fail-fast strategy, where any assertion failure or unhandled exception encountered during the execution of a model element immediately terminates the entire MBT execution. Once such a failure occurs, model traversal stops, no further nodes or edges are explored, and the test run is marked as failed.

This behavior is largely inherited from conventional automated testing frameworks, which assume a linear test structure where a failure invalidates the remainder of the test. However, MBT executions are inherently state-based and exploratory, with test engines dynamically selecting paths to achieve coverage objectives rather than validating a single predefined sequence.

In practice, fail-fast behavior is implemented through rigid exception-handling mechanisms in MBT execution engines. Any failure—regardless of its cause or severity—causes the execution to transition into a terminal state, preventing further traversal. As a result, failures arising from critical defects, minor verification mismatches, or transient execution conditions are treated uniformly.

While this approach simplifies tool design, it fails to exploit the structural properties of MBT models, such as alternative paths and partially independent subgraphs. In large or long-running MBT executions, a single localized failure—possibly occurring late in the traversal—can prematurely terminate exploration of significant portions of the model. The practical implications and limitations of this behavior are discussed in the next subsection.

As an example, this fail-fast behavior is explicitly implemented in GraphWalker through its FailFastStrategy class. As shown in the tool's core execution logic (Fig. 5), any MachineException raised during model execution immediately marks the current model element (node or edge) as failed, sets the execution status to FAILED, and rethrows the exception, thereby terminating the entire MBT run. The strategy does not differentiate between transient, localized, or critical failures; all assertion failures are treated as fatal, and no further model traversal is permitted once an exception occurs.

2.3.4. *Limitations of the fail-fast mechanism in our industrial MBT setting*

In the preceding subsections, we discussed the limitations of the fail-fast mechanism of online MBT tools from a general standpoint. To better understand the practical impact of this behavior, we also assessed the issue in our own industrial setting, which is described in more detail in Section 5.2.

As part of this assessment, we planned and conducted an initial exploratory phase [14], in which the MBT tool GraphWalker was applied in several ongoing industrial test-automation projects. During this exploratory use of online MBT, it was natural for some test steps and assertions in the MBT models to fail, reflecting both transient execution issues and genuine faults in the system under test.

Our observations confirmed that the fail-fast mechanism constitutes a major hindrance to MBT adoption in practice. In all observed cases, test execution was immediately terminated upon encountering an assertion failure. In other words, once a single fault was detected during model execution, the MBT engine halted execution entirely, and the remaining parts of the model were not exercised. As a result, test generation and traversal stopped prematurely, even when the failure was localized and unrelated to other parts of the model.

This strict failure-handling behavior significantly reduces the practicality and robustness of online MBT in large-scale or long-running test scenarios. In particular, it limits achievable coverage, wastes execution effort, and delays feedback on unaffected parts of the system. These observations from our industrial setting reinforce the need for fault-tolerant execution mechanisms in online MBT and directly motivate the heuristics proposed in this paper.

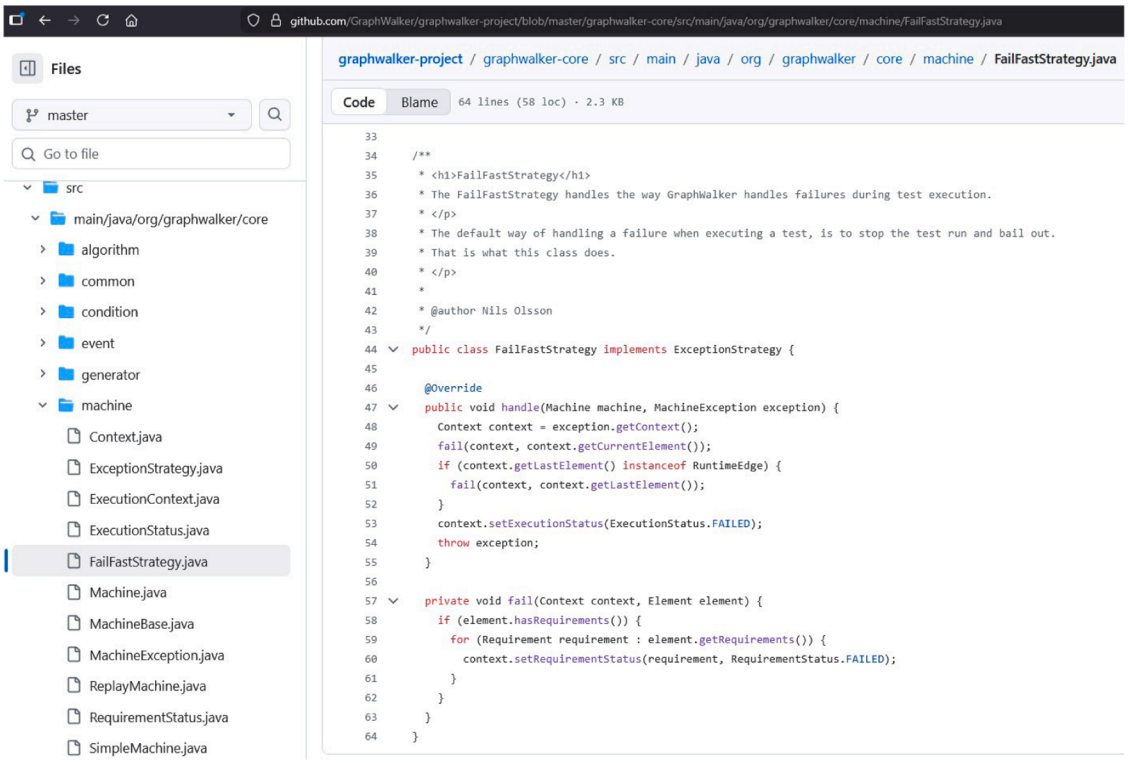


Fig. 5. FailFastStrategy class of the MBT tool GraphWalker[2].

2.4. Research gap and contributions

While prior studies have explored model-based testing execution, tool architectures, and industrial applications, failure-handling semantics in online MBT have received limited explicit attention. In particular, existing tools typically adopt fail-fast execution strategies, and the impact of this behavior on model exploration continuity has not been systematically addressed.

This study makes three contributions: (1) a formalization of fault-tolerant execution extensions for online MBT; (2) tool-level implementation support; and (3) empirical evaluation in an industrial context. By focusing specifically on execution resilience rather than test generation or modeling techniques, the work addresses a previously under-explored dimension of online MBT practice.

3. DESIGN OF THE fault-tolerance heuristics FOR online MBT

We present the design of the two MBT fault-tolerance heuristics below.

3.1. Heuristic 1: retry failed node (Configurable retry mechanism)

As discussed above, online MBT tools typically enforce fail-fast execution semantics, where any assertion failure immediately terminates the ongoing test run. While effective for detecting persistent faults, this behavior can be overly restrictive in industrial testing environments, particularly for UI-centric systems. In such systems, assertion failures may occur due to transient conditions—such as delayed rendering, asynchronous loading, or temporary unavailability of UI elements—rather than genuine functional defects. These failures are commonly referred to as flaky, as they may disappear when the same test logic is re-executed without any changes to the system under test.

The first fault-tolerance heuristic proposed in this paper addresses this issue by allowing a bounded number of retries for a failed model node during online MBT execution. Instead of terminating execution immediately after an assertion failure, the heuristic enables the MBT engine to re-execute the failed node once, thereby providing an opportunity to distinguish transient failures from persistent ones while continuing model traversal when possible.

The retry mechanism is implemented as a configurable parameter that allows a bounded number of reattempts k for a failed node. Rather than fixing this parameter to a single value, we treat the retry bound as an explicit execution-control parameter whose impact can be empirically analyzed. Increasing the value of k would increase the probability of recovering from transient failures but also increases execution overhead due to repeated re-execution of the same transition and node. In the industrial empirical study to be reported in this work (Section 5), we evaluate the retry mechanism using bounded values $k \in \{1, 2, 3, 4, 5\}$ in order to quantify the trade-off between recovery effectiveness and execution cost. The retry bound is kept static during each execution run; adaptive or dynamic adjustment based on historical failure data was considered beyond the scope of this work and is left for future research.

3.1.1. Design rationale

The heuristic operates at the level of model execution and affects only the node at which the failure occurred. When an assertion failure is detected during the execution of a node, the heuristic checks whether the same node has already failed earlier in the current test run. If this is the first failure of the node, the heuristic initiates a retry; if the node has already failed once, execution is terminated and the default fail-fast behavior is applied.

To perform the retry, execution is rolled back to the previously visited node, and the connecting edge that led to the failed node is re-executed. This causes the MBT engine to traverse the same transition again and re-execute the failed node and its associated assertions. By limiting the retry to the immediately preceding transition and node, the heuristic avoids replaying longer execution paths and keeps the retry localized.

3.1.2. Handling execution-state and edge-level issues

Re-executing the connecting edge may itself be affected by execution timing or system state. For example, the action associated with the edge may not have completed fully during the previous attempt. By explicitly allowing the same edge to be re-executed, the heuristic provides additional time for the system under test to reach a stable state before the assertion is evaluated again.

From an execution-engine perspective, assertion failures typically place the MBT engine in a terminal state that prevents further traversal. To enable the retry, the heuristic resets the execution status to an active state after handling the exception, allowing the engine to proceed with re-executing the edge and node.

3.1.3. Bounding the retry

To avoid unbounded retries and potential execution loops, the heuristic uses a bounded retry count *k*per failing node. Each node is marked upon its first failure, and the execution engine keeps track of how many retry attempts have already been applied to that node during the current execution. If the node continues to fail after the configured retry bound is exhausted, execution is halted and the failure is reported in the same manner as under the original fail-fast strategy.

3.1.4. Illustrative example

To illustrate the retry heuristic, we use the MBT login model shown in Fig. 3. In this model, the edge *e_Valid_Login* represents a successful login action that transitions the system from the *Login* page to the *Dashboard* page, where assertions verify that the dashboard is correctly displayed.

Under default fail-fast behavior, if execution of the edge *e_Valid_Login* fails—either because the login action does not complete successfully or because the subsequent verification of the dashboard page fails—the entire MBT execution is terminated immediately. With the proposed retry heuristic, such a failure triggers a rollback to the *Login* page, followed by re-execution of *e_Valid_Login* and the associated dashboard verification, up to the configured retry bound. This behavior allows the MBT engine to tolerate transient issues, such as slow page loading or delayed UI rendering, which may cause a flaky failure on an earlier attempt but succeed in a later retry. If the login or dashboard verification fails again during this retry, execution is terminated as usual.

Once we present the heuristic #2 in the next sections, and when that second heuristic is enabled, execution will not be terminated for the above case; instead, the failed *Dashboard* node is marked as blacklisted and model traversal continues via alternative paths, as discussed in the next subsection.

3.2. Heuristic 2: blacklist the failed node and continue traversal of the MBT model

While the retry heuristic addresses transient failures by allowing a failed node to be re-executed once, some failures persist even after a retry. In online MBT, terminating execution in such cases can significantly reduce model coverage, especially when failures are localized to a specific part of the model and do not invalidate the remainder of the system behavior. This issue is particularly relevant in large MBT models, where stopping execution due to a single failing node may prevent exploration of many unrelated states and transitions.

The second fault-tolerance heuristic addresses this limitation by excluding persistently failing nodes from further traversal while allowing the MBT execution to continue. Instead of terminating execution after a repeated failure, the heuristic marks the failed node as blacklisted and proceeds with model traversal using alternative paths, where available.

3.2.1. Design rationale

The goal of this heuristic is to enable continued exploration of an MBT model when failures are localized and repeatedly occur at the same node. In online MBT, traversal decisions are made dynamically during execution; therefore, excluding a persistently failing node can allow the test engine to reach other parts of the model that remain executable and informative.

Conceptually, this behavior is analogous to route selection when driving in a city. If a driver encounters a road that is closed due to construction or blocked by police, the sensible response is not to stop the journey entirely, but to take an alternative route to reach other destinations. Modern navigation systems similarly detect blocked roads and automatically reroute the driver along available paths. The blacklist heuristic applies the same principle at the level of MBT execution: when a model node repeatedly fails, it is treated as temporarily unavailable, and traversal continues via alternative paths in the model.

From an execution perspective, when an assertion failure occurs at a node that has already failed once (i.e., after the retry heuristic has been exhausted), the blacklist heuristic intervenes as follows. The failed node is marked as blacklisted for the remainder of the execution, the execution context is restored to a valid preceding node, and the MBT engine continues traversal by selecting outgoing edges that do not lead to the blacklisted node. This allows exploration of other model elements without revisiting the failing node.

Importantly, blacklisting does not suppress or ignore failures. The failed node and its assertions are still recorded as failed in the test results. The heuristic only affects traversal decisions, not fault reporting. By doing so, it improves execution continuity and model coverage while preserving full visibility of detected faults.

3.2.2. Illustrative example

Returning to the MBT login model shown in Fig. 3, let us consider again the edge *e_Valid_Login*, which transitions the system from the *Login* page to the *Dashboard* page. If execution of *e_Valid_Login* or verification of the *Dashboard* page fails repeatedly—i.e., the failure persists after the retry heuristic has been applied—the blacklist heuristic is activated.

In this case, the *Dashboard* node is marked as blacklisted, indicating that it should not be visited again during the current execution. Rather than terminating the test run, the MBT engine returns to the *Login* page and continues traversal via other outgoing edges, such as those leading to alternative authentication paths (e.g., third-party login flows) or other login-related scenarios. This allows the MBT execution to continue exploring the remaining parts of the model, even though one specific path has been identified as faulty.

Through this behavior, the blacklist heuristic ensures that a persistent failure in one model element does not prevent the execution and assessment of other, independent parts of the system.

4. Implementation of the fault-tolerance heuristics in mbt tool graphwalker

We describe in this section the implementation of the two proposed fault-tolerance heuristics (Section 3) and integrate of those new code to the large existing code-base of the open-source MBT tool GraphWalker. The implementation was designed to integrate with GraphWalker's existing execution architecture, reuse its object-oriented extensibility mechanisms, and avoid intrusive changes to the core model representation or traversal logic in the tool.

4.1. Need for reverse engineering and program comprehension of the existing graphwalker code-base

Since the goal of this work was to introduce fault-tolerant execution behavior into an existing, mature MBT tool, the implementation was carried out as brownfield software development, rather than greenfield development. GraphWalker is a large and evolving open-source code base, with a non-trivial execution engine, multiple abstraction layers, and well-established execution semantics. Consequently, a thorough understanding of the existing architecture was a prerequisite for

implementing the proposed heuristics in a correct and non-intrusive manner.

To this end, we conducted systematic reverse engineering and program comprehension activities prior to and during the implementation phase. These activities focused on understanding the execution flow of online MBT runs, the propagation of assertion failures through the execution engine, and the interaction between traversal logic, execution state, and exception handling. Particular attention was paid to identifying suitable extension points where fault-tolerance behavior could be introduced without breaking existing functionality. At the time of our technical work on this project, the latest version of the GraphWalker code-base was version 4.3.1. It had 283 Java files (classes) with total LOC of 30,162.

We employed static analysis techniques, including dependency and call-graph analysis, using the IntelliJ Java IDE. We also carefully studies the system's reverse-engineers class diagram to ensure fully understanding where to add our extended classes in the code base. These analyses enabled us to trace how execution control flows through the core classes of the GraphWalker engine, identify classes responsible for managing execution state and failure handling, and assess the potential impact of code changes. This program comprehension effort informed both the design of the new execution strategies and the identification of the minimal set of existing classes that needed to be modified to support fault-tolerant execution.

To support interested readers in program comprehension of the code-base, we provide several reverse-engineered artifacts in the GitHub repository[4] of our work, e.g., class diagrams and dependency graphs.

4.2. Overall implementation strategy and design considerations

The implementation strategy was guided by several key considerations. First, fault tolerance was implemented at the execution layer, not at the model-definition level. This design choice ensures that existing MBT models, test code, and modeling practices remain unaffected. Test engineers can therefore adopt the proposed heuristics without modifying their models or test logic.

Second, the implementation builds on GraphWalker's existing execution and exception-handling mechanisms. Our implementation of the heuristics selectively extended the existing mechanism in the MBT tool by intervening only after execution failures occur. This preserves the default semantics of the tool and limits behavioral changes to well-defined situations.

Third, the two heuristics were implemented as explicit, separate execution strategies. This separation improves clarity, supports independent activation of the heuristics, and simplifies reasoning about their behavior. It also facilitates future extensions, such as introducing additional execution strategies or making heuristic selection configurable at runtime.

4.3. Integration with graphwalker's exception strategy framework

GraphWalker manages execution-time failures through an exception strategy abstraction that governs how the execution engine reacts to assertion failures and runtime exceptions. Under the default configuration, this mechanism enforces strict fail-fast semantics: the first assertion failure immediately terminates execution.

The proposed heuristics were implemented by extending this framework with two new exception strategies:

- TryAgainStrategy, which implements the retry-failed-node heuristic, and
- BlackListStrategy, which implements the blacklist-failed-node heuristic.

[4] github.com/vgeruslu/Fault-tolerance-for-MBT

By integrating at this level, the heuristics are applied precisely at the point where execution failures are detected and handled. Each strategy inspects the execution context, evaluates the execution history of the failing node, and decides whether execution should terminate, retry the node, or continue traversal via alternative paths. This approach avoids duplicating execution logic and ensures compatibility with existing traversal and execution mechanisms.

4.4. Execution context extensions and node-level state tracking

To enable the proposed fault-tolerance heuristics, the execution engine must be able to reason about the *execution history of individual model nodes*. However, prior to this work, GraphWalker did not provide an explicit mechanism to track the execution status of nodes within an MBT model during a test run. In particular, the tool did not maintain information about whether a node had already been executed successfully, had failed, or should be excluded from further traversal. Such information is essential for distinguishing between first-time failures, repeated failures, and nodes that should no longer be visited.

To implement this capability, we introduced a **simple, explicit node-level status abstraction** and integrated it into the execution context. Each node (vertex) in the MBT model is associated with a status that reflects its execution outcome during the current test run. This status is initialized when execution starts and is updated dynamically as execution progresses.

The following node states are tracked:

- NOT_COVERED: the node has not yet been executed (default state),
- COVERED: the node has been executed successfully without assertion failures,
- FAILED: the node has encountered an assertion failure during execution,
- NOT_REACHABLE: the node has been marked as excluded from further traversal due to blacklisting.

This lightweight state representation was realized using a simple enumeration structure (enum) in Java and integrated into the execution context. The choice of an enum-based approach kept the implementation straightforward, avoided unnecessary complexity, and made node status transitions explicit and easy to reason about. The corresponding code listing of the new Java enumeration type is shown in Fig. 6.

The retry heuristic relies on this state information to determine whether a failure is occurring for the first time or is a repeated failure, thereby deciding whether a retry is permitted. The blacklist heuristic, in turn, uses the **NOT_REACHABLE** state to dynamically exclude persistently failing nodes from traversal while allowing execution to continue elsewhere in the model. Importantly, this state tracking is scoped to a single MBT execution and does not modify the underlying model or persist across test runs.

This explicit state tracking allowed the execution engine to distinguish between first-time failures and repeated failures, and to dynamically adjust traversal behavior accordingly. The retry heuristic relies on this information to determine whether a retry is permitted, while the blacklist heuristic uses it to exclude persistently failing nodes from future traversal. All state information is scoped to a single execution and does not persist across runs, ensuring that each test execution starts from a clean state.

4.5. Implementation of heuristic 1: retry failed node

The retry heuristic is implemented in the *TryAgainStrategy* class. When an assertion failure occurs during node execution, the strategy consults the node's execution status to determine whether the node has failed previously.

If the node has not failed before, the strategy initiates a retry by:

```java
/**
 * NodeStatus
 * Each NodeStatus is associated with a vertex/node which keeps track of its status.
 * The default status is NOT_COVERED.
*/
public enum NodeStatus {
  NOT_COVERED, COVERED, FAILED, NOT_REACHABLE
}
```

Fig. 6. A new Java enumeration type defined to is enable node-level state tracking, during test execution of a given MBT model.

1. recording the failure in the execution context,
2. restoring the execution engine to an active execution state,
3. rolling execution back to the immediately preceding node,
4. scheduling re-execution of the connecting edge that led to the failed node.

This causes the MBT engine to re-traverse the same transition and re-execute the failed node and its associated assertions. If the node fails again during this retry attempt, the strategy ceases intervention and allows the default fail-fast behavior to terminate execution. This bounded behavior ensures that transient failures can be tolerated while persistent faults are still detected promptly.

The retry count k is exposed as a configuration parameter in the execution strategy. For the empirical study, we varied k from 1 to 5 systematically to assess how increasing the retry bound influences recovery from transient failures and associated execution overhead.

4.6. Implementation of heuristic 2: blacklist failed node and continue traversal

The blacklist heuristic is implemented in the *BlackListStrategy* class and is activated when a node fails persistently, i.e., after the retry heuristic has already been applied.

When this heuristic is triggered, the failing node is marked as NOT_REACHABLE in the execution context. Execution is then restored to a valid preceding node, and model traversal continues while excluding the blacklisted node from future selection. Traversal decisions are made using the existing path-generation logic, subject to the constraint that blacklisted nodes are not revisited.

This approach allows exploration of unaffected parts of the model even when certain nodes consistently fail. Importantly, blacklisting does not suppress failure reporting: failed nodes remain recorded as failed in the test results. The heuristic therefore improves execution continuity and model coverage without hiding detected faults.

Java source codes of both classes *TryAgainStrategy* and *BlackListStrategy* are provided in Appendix A.

4.7. Performance overhead considerations

The proposed heuristics introduce minimal computational overhead to the MBT execution engine. Both mechanisms operate through simple conditional checks integrated into the exception-handling path of the execution engine. The retry heuristic maintains a small counter associated with the current node and performs a constant-time conditional check to determine whether the retry bound has been reached.

Similarly, the blacklist heuristic performs a constant-time membership check to determine whether a node has been marked as blacklisted before traversal continues. These operations are implemented using lightweight in-memory data structures and involve only simple control-flow logic (e.g., conditional checks and counter updates). Since the dominant cost in online MBT execution arises from interactions with the SUT (e.g., UI actions and assertion evaluations), the additional control logic introduced by the heuristics contributes negligible runtime overhead relative to the overall execution time.

4.8. Summary of implementation, code modifications, and replicability artifacts

This section summarizes the concrete implementation changes made to the GraphWalker code base and provides pointers to the publicly available artifacts supporting transparency and replicability.

To implement the proposed fault-tolerance heuristics, we developed and added **three new classes** to the GraphWalker code base, all located under the graphwalker/core/machine package:

- NodeStatus.java: introduces an explicit node-level execution status abstraction used to track execution outcomes during a test run.
- TryAgainStrategy.java: implements the retry-failed-node heuristic.
- BlackListStrategy.java: implements the blacklist-failed-node heuristic.

In addition to introducing these new classes, a small number of **existing core classes** were modified to integrate the new functionality and make the heuristics operational within the execution engine. These changes were intentionally limited in scope and confined to execution-time components:

- SimpleMachine.java (graphwalker/core/machine): extended to support alternative exception-handling strategies.
- ExecutionContext.java (graphwalker/core/machine): extended to store and manage node-level execution state.
- Context.java (graphwalker/core/machine): updated to propagate execution-state information consistently.
- MachineBase.java (graphwalker/core/machine): adapted to enable controlled continuation of execution after failures.
- Vertex.java (graphwalker/core/model): minimally extended to support execution-status tracking at the model-element level.

All newly developed classes and the modified versions of the existing classes are available as open-source code in a public repository hosted on GitHub: github.com/vgeruslu/Fault-tolerance-for-MBT

Further details about the design rationale, class interactions, and code-level modifications are documented in a comprehensive design and implementation document included in the repository.

To support understanding of the runtime behavior of the two heuristics, a demo video illustrating both fault-tolerance mechanisms in action during MBT execution is also publicly available: youtu.be/6vSwHN7UDO4

All the above provided artifacts would enable other researchers and practitioners to inspect the implementation, reproduce the experiments, and extend the proposed fault-tolerance mechanisms in their own MBT environments.

5. Industrial case-study

5.1. Goal and research questions

The empirical evaluation in this study is guided by the Goal–Question–Metric (GQM) approach. The overarching goal is to evaluate the effectiveness of the proposed fault-tolerant features for online

Model-Based Testing (MBT) execution, as implemented in the GraphWalker tool, in comparison to the default fail-fast execution behavior.

More specifically, the goal of the evaluation is to assess whether and to what extent the proposed heuristics improve the robustness and practicality of online MBT execution in industrial testing scenarios, without masking genuine faults or introducing undue overhead.

Based on this goal, we formulate the following research questions:

- **RQ1:** How effective is the feature based on Heuristic 1 (Retry Failed Node) in handling transient failures during online MBT execution, compared to the default fail-fast behavior of the MBT tool?
- **RQ2:** How effective is the feature based on Heuristic 2 (Blacklist Failed Node) in enabling continued traversal of the MBT model after persistent failures, compared to the default fail-fast behavior of the MBT tool?

5.2. Industrial case-study context: MBT at testinium A.Ş

The empirical evaluation was conducted in collaboration with Testinium A.Ş., where online Model-Based Testing (MBT) has been applied in industrial projects since 2019 using the GraphWalker tool. The organization has accumulated several years of practical experience with large-scale online MBT execution in web-based systems [14].

The goal of this case study is to investigate the behavior of the proposed fault-tolerant execution heuristics in a realistic industrial MBT environment. Conducting controlled experiments across multiple industrial systems under test (SUTs) was not feasible in the scope of this study due to the effort required to construct and maintain MBT models and automated test suites for each additional system. Instead, we focused on a single industrial SUT with a non-trivial MBT test suite, enabling controlled experimentation and detailed analysis of execution behavior under different failure scenarios. This approach follows common practice in empirical software engineering, where an in-depth case study on a representative industrial system is used to evaluate the feasibility and practical impact of a proposed technique.

The specific focus of this case study is the behavior of online MBT execution under assertion failures, and the effectiveness of the proposed fault-tolerance heuristics in improving execution continuity. The industrial setting provides a realistic environment in which online MBT is used for testing UI-intensive systems, where transient and persistent execution failures may occur during long-running test sessions.

The following subsection describes the System Under Test (SUT) and the MBT models used for the evaluation.

5.3. System under test (SUT)

The System Under Test (SUT) used in this study is *Testinium* (testinium.io), the company's web-based test-management platform. This same SUT was also previously studied in one of our previous industrial MBT experience papers [14]. Using the same SUT and MBT test suite allows this study to build directly on an established and well-understood industrial setup. For full experimental reproducibility, the MBT test suites of the SUT are publicly available as open-source artifacts: github.com/vgeruslu/MBTofTestinium

Testinium is a modern web application with multiple UI pages, asynchronous interactions, and external integrations. Following MBT best practices, a modular modeling approach was used, with a separate MBT model developed for each major UI page. In total, the SUT comprises 18 UI pages, resulting in 18 MBT models with 177 nodes and 260 edges across the full test suite [14]. As a reminder, nodes represent user-visible states and verification points, while edges represent user actions such as navigation, form submission, and authentication steps.

Assertions are associated with nodes and implemented using Selenium-based Java code, enabling fully automated online MBT execution. The size and structure of the MBT models, together with the dynamic nature of the web application, make this SUT well suited for evaluating fault-tolerant online MBT execution, where assertion failures—both persistent and transient—may occur during long-running test sessions.

5.4. Evaluation approach and setup

To evaluate the proposed fault-tolerance heuristics in a controlled and repeatable manner, we employed controlled fault injection during online MBT execution. Ideally, faults would be injected directly into the System Under Test (SUT). However, modifying the production-grade Testinium platform was not feasible in our industrial environment. Instead, failures were injected by modifying assertions in the MBT test suite, causing them to fail at predefined points during model traversal while leaving the SUT unchanged. By altering assertions so that they fail under otherwise normal execution, we were able to trigger assertion failures deterministically at known points during model traversal, while keeping the SUT itself unchanged.

From the perspective of online MBT execution, this approach is sufficient for evaluating execution-level fault-tolerance behavior. In online MBT, assertion failures—regardless of whether they originate from SUT defects or incorrect assertions—are handled uniformly by the execution engine. An assertion failure simply signals a violation of expected behavior, and the execution engine reacts accordingly. Therefore, injecting assertion-level faults provides an appropriate mechanism for evaluating retry and blacklisting behavior.

5.4.1. Injected fault locations

Five assertion faults were injected into the login MBT model (Fig. 3) at explicitly identified verification nodes. The injected locations, failure characteristics, and their intended role in evaluating the two heuristics are summarized in Table 1.

In brief, four of the injected faults (mutations 1, 2, 3, and 5) were designed as deterministic persistent failures, meaning that the modified assertion fails on every execution attempt. These mutations are primarily used to evaluate the effectiveness of the Blacklist Failed Node heuristic in enabling continued traversal after persistent localized failures.

The remaining mutation (mutant 4, in the vertex *v_Verify_In_Dashboard_Page_SHARED*) was intentionally implemented as a non-deterministic transient failure, where the assertion fails with probability 0.5 on each attempt. This probabilistic behavior allows controlled evaluation of the Retry Failed Node heuristic under varying retry bounds. In other words**, to create** mutant 4. the assertion verifying successful navigation to the dashboard page after a valid login (via edge *e_Valid_Login*) was modified to include a simple code block using the random() function, causing it to pass or fail with a 50% probability. This modification simulated non-deterministic and transient failures that may occur in a faulty deployment environment. The injected fault also represented realistic post-login verification issues, such as incomplete page loading or delayed rendering of dashboard components.

As shown in Table 1, the selected injection points span structurally diverse regions of the MBT model, including initial navigation states, validation states, post-authentication verification, and auxiliary authentication flows. This distribution ensures that failures occur at different traversal depths and connectivity patterns within the model, enabling meaningful assessment of execution continuity under both heuristics.

The five injected faults were designed to evaluate the two heuristics under distinct failure conditions:

- The four deterministic mutations (1, 2, 3, and 5) always fail when executed. Since retrying does not change the outcome, these mutations primarily assess Heuristic 2 (Blacklist Failed Node) by evaluating whether execution continues beyond persistent localized failures.

Table 1
Injected faults and their role in evaluating the two heuristics.

#	Fault injected in which vertex / node (Mutation)	Failure Type	Deterministic / Non-Deterministic	Primary Heuristic Evaluated	Rationale
1	v_Verify_In_Login_Page	UI verification failure	Deterministic	Heuristic 2 (Blacklist)	Always fails; retry cannot resolve it. Used to assess whether execution continues after persistent failure via blacklisting.
2	v_Verify_Invalid_Password	Validation feedback failure	Deterministic	Heuristic 2 (Blacklist)	Persistent assertion failure; retry expected to fail again. Evaluates isolation and continued traversal behavior.
3	v_Verify_Invalid_Login_Empty	Input validation failure	Deterministic	Heuristic 2 (Blacklist)	Early-stage persistent failure. Assesses whether traversal continues when failure occurs near root of model.
4	v_Verify_In_Dashboard_Page_SHARED	Post-login verification	**Non-deterministic (50% random)**	**Heuristic 1 (Retry)**	Designed to simulate transient/flaky failure. Retry may succeed on second attempt, directly evaluating recovery capability.
5	v_Verify_In_Forgot_Password_Page	Navigation verification failure	Deterministic	Heuristic 2 (Blacklist)	Persistent failure in auxiliary flow; evaluates whether alternative paths remain explorable.

- The probabilistic mutation (#4 in Table 1) was intentionally designed to evaluate Heuristic 1 (Retry Failed Node). Because each execution attempt has a 50% chance of failure, retrying may result in eventual success. This allows quantitative analysis of recovery probability under bounded retry counts.

This separation ensures that each heuristic is evaluated under conditions aligned with its intended purpose: retry for transient failures, and blacklisting for persistent failures.

5.4.2. Retry-Bound sensitivity analysis

To analyze the behavior of the retry heuristic, the probabilistic mutation (#4) was executed under varying retry bounds k. The retry count is implemented as a configurable parameter in the execution strategy. For the evaluation, we used bounded values: $k \in \{1, 2, 3, 4, 5\}$. For each retry bound, MBT execution was repeated multiple times under identical configuration. We recorded: (1) whether execution continued beyond the failure point, and; and (2) the additional execution overhead introduced by retries.

Under the assumption of independent 50% failure probability per attempt, the theoretical probability of eventual success within $k$attempts is:

$$P(\text{success within } k) = 1 - 0.5^k$$

This provides a reference baseline for interpreting empirical continuation rates and allows assessment of the trade-off between recovery probability and execution overhead.

5.4.3. Selection rationale and bias mitigation

The five injection points were selected to cover structurally diverse regions of the MBT model, including:

- initial navigation states,
- input-validation states,
- post-authentication verification states,
- auxiliary authentication flows.

The selected nodes differ in traversal depth and graph connectivity (in-degree and out-degree), ensuring that failures affect model traversal in different ways.

We intentionally avoided injecting faults exclusively in terminal nodes (i.e., nodes with no outgoing transitions), since their failure would not influence further traversal. We also avoided injecting faults only in structurally isolated nodes that participate in narrow execution paths. Instead, the chosen nodes are integrated into the model's traversal structure, enabling meaningful assessment of continued exploration under both heuristics.

An exhaustive injection across all 177 nodes of the MBT suite was not conducted, as the objective of this study is to evaluate execution-control semantics rather than node-specific recovery characteristics. Since the heuristics operate at the execution-engine level and are independent of domain-specific node semantics, their behavior depends primarily on traversal topology rather than functional meaning.

5.5. Empirical results and analysis

We present in this section empirical results of RQ1 and RQ2 and analyze them next.

5.5.1. Experimental execution and measurement approach

Section 5.4 described the controlled fault-injection setup used to evaluate the proposed fault-tolerance heuristics. In this section, we report the empirical results obtained under that experimental configuration. The MBT executions were conducted using the GraphWalker-based test infrastructure described earlier (Section 2.3.1), with injected faults triggering assertion failures at predefined model nodes.

For the probabilistic mutation (*v_Verify_In_Dashboard_Page_SHARED*, #4 in Table 1), which fails with probability 0.5 on each execution attempt, multiple MBT runs were executed under varying retry bounds k. The retry count was varied within the range $k \in \{1, 2, 3, 4, 5\}$ in order to analyze the impact of retry depth on recovery from transient failures and the associated execution overhead. Each configuration was executed repeatedly under identical conditions to observe continuation behavior beyond the failure point.

For the remaining deterministic mutations (mutations 1, 2, 3, and 5 in Table 1), the injected assertions always fail when executed. These faults were used primarily to assess the behavior of the blacklist heuristic, which isolates persistently failing nodes and enables continued traversal of alternative paths in the MBT model.

The evaluation focused on the following observable outcomes / criteria:

- Execution continuation after transient failures: whether model traversal can continue beyond a failing node when bounded retries are applied (Heuristic 1).
- Traversal robustness under persistent failures: whether the blacklist heuristic enables the execution engine to bypass persistently failing nodes and continue exploring alternative paths in the model (Heuristic 2).

The empirical findings addressing the two research questions are presented in the following subsections.

5.5.2. RQ 1: effectiveness of the retry heuristic (#1)

To evaluate the effectiveness of the retry heuristic, we focused on mutation #4. As described in Section 5.4, this mutation was intentionally implemented as a probabilistic failure using a random() function. The assertion fails with probability 0.5 on each execution attempt. This setup simulates transient failures that may occur in real testing environments, for example due to delayed UI rendering, incomplete page

loading, or other timing-related issues.

Under the default fail-fast execution strategy (i.e., without retry), execution terminates immediately when the assertion fails. Since the mutation fails with probability 0.5, approximately half of the runs terminate prematurely at this node, preventing further model traversal.

When the retry heuristic is enabled, the execution engine is allowed to reattempt the failing node up to a bounded retry count k. If any retry attempt succeeds, execution continues normally. This mechanism allows transient failures to be tolerated without prematurely stopping the MBT exploration.

To analyze the impact of the retry bound, we executed the MBT model repeatedly while varying the retry count k. For each configuration, the test execution was repeated multiple times (we set that max to 20) under identical conditions, and we recorded whether the execution successfully continued beyond the failing node. Table 2 summarizes the observed results.

Because the injected failure has a probability of 0.5 on each attempt, the theoretical probability of eventual success within $k$attempts can be calculated as: $P(success) = 1 - (0.5)^k$. This theoretical expectation is shown in the final column of Table 2 for reference.

Table 2 reports the empirical continuation rates observed for different retry bounds and compares them with the theoretical probabilities derived from the probabilistic failure model. The observed continuation rates closely follow the expected trend.

For example, when the retry bound is $k = 1$, the execution engine attempts the failing node only once. In this configuration, the injected assertion has a 50% probability of failure, meaning that approximately half of the runs terminate at this node. In our experiment, out of 20 executions, 11 runs successfully continued beyond the failure point, corresponding to an observed continuation rate of 55%, which is close to the theoretical expectation of 50%.

When the retry bound is increased to $k = 2$, the execution engine allows one additional attempt if the first attempt fails. This means that the node can be executed up to two times before the execution is terminated. In our experiment, 16 out of 20 runs continued successfully, yielding an observed continuation rate of 80%. This result is close to the theoretical continuation probability of 75%, which reflects the increased chance that at least one of the two attempts succeeds.

Minor deviations between observed and theoretical values are expected due to the stochastic nature of the experiment and the relatively small sample size (20 runs per configuration). For example, the continuation rate for $k = 3$was observed as 85%, slightly below the theoretical value of 87.5%. Such variations are consistent with random sampling fluctuations and do not affect the overall trend. As the retry bound increases, the continuation probability consistently increases, demonstrating that bounded retries significantly improve the likelihood that execution continues beyond transient failures.

The results also illustrate diminishing returns for larger retry bounds. While increasing the retry bound from $k = 1$to $k = 3$substantially improves continuation probability, further increases provide only marginal improvements relative to the additional execution overhead introduced by repeated attempts.

These findings confirm that bounded retries provide an effective mechanism for handling transient failures during online MBT execution.

Table 2
Retry heuristic evaluation results.

Retry Bound (k)	Total Number of MBT Test Runs	Number of MBT Runs Continuing After Failure	Observed Continuation Rate (%)	Theoretical Continuation Probability
1	20	11	11 / 20 = 55%	50%
2	20	16	16/ 20 = 80%	75%
3	20	17	17/20=85%	87.5%
4	20	19	19/20=95%	93.75%
5	20	19	19/20=95%	96.9%

At the same time, the results illustrate a practical trade-off: larger retry bounds improve recovery probability but also increase execution time due to repeated attempts. For this reason, the retry count is implemented as a configurable parameter so that practitioners can select an appropriate value depending on the expected level of test flakiness and acceptable execution overhead.

5.5.3. RQ 2: effectiveness of the blacklist heuristic (#2)

To evaluate the effectiveness of the blacklist heuristic, we analyzed the behavior of the MBT execution under the four deterministic mutations introduced in Section 5.4 (mutations 1, 2, 3, and 5). In these mutations, the injected assertions always fail when executed. Therefore, retries cannot resolve the failure because the outcome of the assertion is deterministic.

Under the default fail-fast execution strategy used in most online MBT tools, including GraphWalker, execution terminates immediately when such an assertion failure occurs. Consequently, model exploration stops at the failing node, preventing traversal of the remaining reachable parts of the model.

The blacklist heuristic modifies this behavior by isolating the failing node after the failure is detected. When a failure occurs, the corresponding node is recorded in a blacklist and excluded from further traversal attempts during the current execution session. The traversal algorithm then continues exploring alternative paths in the model that do not include the blacklisted node.

To empirically evaluate the correctness and robustness of this mechanism, we applied two evaluation criteria. First, we verified whether the MBT execution **continues after encountering the failure**, rather than terminating as in the fail-fast baseline. Second, we verified whether the execution engine **never revisits the blacklisted node during the same execution run**. The latter property is essential to ensure that the blacklist mechanism effectively prevents repeated failures at the same location.

To verify the second criterion, we instrumented the execution monitoring logic to record visits to each model node during traversal. When a node was added to the blacklist due to a failure, the monitoring mechanism checked whether that node was ever selected again by the traversal algorithm.

To further verify that the blacklist mechanism enables meaningful continuation of model exploration, we examined the execution traces after the failure point. In each of the four deterministic mutation scenarios, once the failing node was blacklisted, the traversal algorithm continued executing additional model transitions along alternative paths. The execution logs confirmed that multiple nodes and edges beyond the failure point were successfully traversed, demonstrating that the blacklist heuristic enables continued state-space exploration rather than merely preventing repeated failures.

The results confirmed that the blacklist heuristic behaves as intended. When a deterministic failure occurs, the failing node is isolated and excluded from further traversal attempts, allowing the execution to continue exploring other reachable parts of the model without repeatedly encountering the same failure. This behavior significantly improves the robustness of online MBT execution in the presence of persistent faults.

While the blacklist mechanism does not resolve the underlying defect, it ensures that a single persistent failure does not prematurely terminate the entire test session and allows the remaining parts of the model to be explored.

5.5.4. Overall impact of fault-tolerant execution

Across the conducted experiments, enabling the proposed fault-tolerance heuristics consistently prevented premature termination of online MBT executions that would otherwise occur under the default fail-fast behavior. In the baseline configuration (without heuristics), execution stopped immediately upon encountering the first assertion failure, preventing further exploration of the model. In contrast, the

retry heuristic enabled recovery from transient failures under bounded retry conditions, while the blacklist heuristic allowed traversal to continue after persistent localized failures.

As a result, the MBT engine was able to explore additional nodes and transitions beyond the failure point, enabling longer test sessions and broader exploration of the model compared to the baseline configuration. These observations demonstrate that introducing controlled fault-tolerant execution can significantly improve the practical continuity of online MBT execution in the evaluated industrial setting.

5.5.5. Execution overhead

We also considered the potential runtime overhead introduced by the proposed heuristics. In practice, the additional logic consists only of lightweight conditional checks and small state updates within the MBT execution engine. Because online MBT executions are dominated by interactions with the SUT (e.g., Selenium-driven UI actions, network requests, and assertion evaluations), the computational cost of these additional checks is negligible in comparison. During the experiments, no observable increase in execution time attributable to the heuristics was detected. Therefore, the proposed mechanisms improve execution robustness without introducing meaningful performance overhead.

6. Discussions

We discuss in this section the implications of the empirical findings presented in Section 5, with a focus on their practical relevance, generalizability, and limitations. We interpret how the proposed fault-tolerance heuristics affect the usability and adoption of online MBT in the industrial context, consider their applicability to other MBT tools, and reflect on the limitations of the study and associated threats to validity.

6.1. Benefits of the approach and its adoption in the industrial context

The primary benefit of the proposed fault-tolerance approach is its impact on the practical operationalization of online MBT in an industrial setting. In the context of Testinium, strict fail-fast execution semantics had become a significant barrier to effective MBT use. In practice, any assertion failure—whether caused by a genuine defect or a transient issue—would immediately terminate execution, preventing the remaining parts of the model from being exercised. As a result, online MBT executions frequently stopped prematurely, reducing coverage and diminishing their practical value.

This behavior led to growing frustration among test engineers, as MBT runs could not proceed unless the detected issue was first fixed in the SUT. Consequently, testing activities became tightly coupled to defect-fixing cycles, and the exploratory and coverage-driven advantages of online MBT were largely lost. In this situation, online MBT risked becoming ineffective for day-to-day industrial testing.

By introducing the retry and blacklist heuristics, online MBT execution became robust to both transient and persistent failures. Transient issues could be handled through bounded retries, while known failing parts of the model could be isolated without blocking traversal of the remaining paths. This enabled long-running MBT executions to complete meaningfully, even in the presence of unresolved defects.

From an adoption perspective, these capabilities had a clear and immediate effect. Online MBT became usable again as a practical testing approach rather than an all-or-nothing activity. Test engineers were able to continue exercising large parts of the model, gain feedback from multiple scenarios in a single execution, and prioritize defect fixing more effectively. As a result, MBT usage within the company increased, and the approach became integrated into regular industrial testing activities rather than being treated as an experimental or fragile technique.

Overall, the proposed heuristics address a fundamental mismatch between strict fail-fast execution and industrial testing realities. By aligning online MBT execution behavior with how testing is conducted in practice, they significantly improve the feasibility and acceptance of MBT in real-world industrial contexts.

6.2. Applicability of the fault-tolerance heuristics to other MBT tools

Although the proposed fault-tolerance heuristics were implemented and evaluated using GraphWalker, their underlying concepts are not tool-specific and can be applied to other MBT tools that support online model execution. Both heuristics operate at the execution-control level and do not depend on GraphWalker-specific modeling constructs.

The Retry Failed Node heuristic requires only the ability to detect assertion failures and re-execute recently traversed transitions, capabilities commonly available in online MBT engines. The Blacklist Failed Node heuristic relies on lightweight execution-state tracking and dynamic exclusion of failing nodes from traversal, which aligns with how many MBT tools already manage coverage and path selection.

Importantly, neither heuristic requires changes to the MBT model or test code, which lowers adoption effort and increases portability. The results of this study demonstrate that introducing fault-tolerant execution semantics can significantly improve the robustness of online MBT execution in the evaluated environment. While the implementation and evaluation were conducted using GraphWalker and a single industrial system under test, the proposed heuristics operate at the execution-control layer of the MBT engine rather than at the modeling or SUT-integration layer. In most online MBT tools, the execution engine is responsible for state traversal, assertion evaluation, and exception handling during model execution. The Retry Failed Node heuristic only requires the ability to detect assertion failures and re-execute recently traversed transitions, while the Blacklist Failed Node heuristic relies on tracking failing nodes and excluding them from subsequent traversal decisions. These capabilities are conceptually present in the execution architectures of many online MBT tools. Therefore, although the empirical evaluation is limited to GraphWalker, the design of the heuristics is intentionally tool-agnostic and compatible with common MBT execution mechanisms. Nevertheless, validating the effectiveness of these mechanisms across additional MBT tools and systems remains necessary to confirm their general applicability.

Unlike traditional retry mechanisms that operate at the test-case level, the proposed heuristics adapt execution behavior at the model traversal level, which allows online MBT executions to continue exploring the model despite localized failures.

6.3. Limitations and threats to validity

The approach presented in this study (Sections 3 and 4) and the empirical investigation reported in Section 5 should be interpreted in light of several limitations and potential threats to validity. In the following, we first discuss limitations of the proposed fault-tolerance approach itself, and then analyze threats to validity of the empirical evaluation using four commonly adopted categories of validity threats—internal validity, construct validity, conclusion validity, and external validity [31–35].

6.3.1. Limitations of the proposed approach

The proposed fault-tolerance heuristics are designed to improve the robustness of online MBT execution, not to replace or weaken fault detection. As such, they only aim at mitigating their impact on continuity of MBT execution. Consequently, when failures are persistent in the SUT, the blacklist heuristic excludes affected nodes from traversal, which may delay testing of those specific behaviors until defects are fixed.

In addition, the approach assumes that failures are localized at the level of individual model nodes or transitions. In scenarios where failures are systemic—affecting large portions of the model or core navigation paths—fault-tolerant execution may still be limited, as many nodes may become unreachable. In such cases, continued traversal

provides diminishing returns, and defect resolution becomes a prerequisite for meaningful testing.

Finally, the heuristics operate at the execution-control level and rely on accurate identification of failing model elements. If failure attribution is ambiguous—for example, when failures arise from shared setup code, external dependencies, or environment instability—retrying or blacklisting individual nodes may not fully capture the underlying cause. Addressing such scenarios may require complementary techniques, such as environment stabilization or enhanced failure diagnostics, which are outside the scope of this study.

6.3.2. Threats to validity of the empirical evaluation

6.3.2.1. Internal validity. A potential threat to internal validity arises from the use of controlled fault injection at the test-suite level rather than injecting real defects into the SUT. Modifying the production SUT was not feasible in the industrial context. To mitigate this threat, assertion faults were injected in a controlled and repeatable manner at known model locations. Since online MBT engines react uniformly to assertion failures regardless of their root cause, this approach is sufficient for evaluating execution-level fault-tolerance behavior.

6.3.2.2. Construct validity. Construct validity concerns whether the chosen measures adequately capture the intended concepts. Effectiveness was assessed through observable execution outcomes such as premature termination, retry behavior, continued traversal, and failure reporting. These measures directly reflect the goals of the proposed heuristics. The study does not assess fault-detection effectiveness or defect severity, which is a deliberate scope decision aligned with the research questions.

6.3.2.3. Conclusion validity. Threats to conclusion validity include the limited number of injected faults and the absence of large-scale statistical analysis. The evaluation relies on comparative execution behavior under controlled conditions (with vs. without heuristics). Using identical models, configurations, and fault locations across runs helps isolate the effect of the proposed features. For RQ1, multiple retry bounds were evaluated under a controlled probabilistic failure model. Although the failure probability was artificially fixed at 0.5 for experimental control, real-world flaky behavior may exhibit different distributions. Nevertheless, the monotonic relationship between retry bound and recovery probability is generalizable to independent transient failure scenarios.

6.3.2.4. External validity. The primary threat to external validity is that the empirical evaluation was conducted using a single industrial System Under Test (Testinium) and a single MBT tool (GraphWalker). Consequently, the observed benefits of the proposed heuristics may not directly generalize to all application domains, system architectures, or MBT frameworks. To partially mitigate this threat, the selected SUT represents a realistic and moderately complex industrial web application with multiple UI pages, asynchronous interactions, and nondeterministic execution behavior—conditions that are typical in many modern automated testing environments. Furthermore, the proposed heuristics were deliberately designed to operate at the execution-control layer of the MBT engine, independent of the modeling language, test code structure, or specific SUT implementation. Because this architectural layer exists in most online MBT tools, the underlying concepts of retry-based recovery and node-level blacklisting are expected to be transferable to other MBT frameworks that support dynamic model traversal.

Nevertheless, the empirical results presented in this paper should be interpreted as initial evidence obtained from a single industrial case study rather than as definitive proof of broad generalizability. Additional empirical studies involving different MBT tools, system domains, and model structures would further strengthen the external validity of the findings. Replicating the evaluation across multiple MBT environments represents an important direction for future work.

In addition, the empirical evaluation relies on a single MBT tool (GraphWalker) and a single industrial SUT (Testinium). While this limits the immediate generalizability of the results, the case study provides an in-depth evaluation using a moderately complex industrial system with multiple models, UI pages, and injected failure scenarios. Furthermore, the proposed heuristics operate at the execution-control layer of the MBT engine and are largely independent of SUT-specific characteristics. Future work should replicate the study across additional SUTs and MBT tools to further evaluate the generality of the proposed approach.

7. Conclusions and future work

This paper addressed a fundamental limitation of online Model-Based Testing (MBT) tools: the strict fail-fast execution behavior that causes test execution to terminate upon the first assertion failure. Through industrial experience, we showed that this behavior significantly reduces the practicality of online MBT and can hinder its adoption in real-world testing environments.

To address this issue, we proposed two lightweight fault-tolerance heuristics—**Retry Failed Node** and **Blacklist Failed Node**—that operate at the execution-control level of an MBT engine. The heuristics were implemented in the GraphWalker tool and evaluated through an industrial case study. The results demonstrate that these heuristics enable online MBT execution to continue in the presence of transient and persistent failures, respectively, while preserving fault visibility and introducing only bounded overhead.

Our findings provide empirical evidence that introducing fault-tolerant execution semantics can improve the robustness and practical usability of online MBT execution in industrial testing environments. In the conducted case study, the default fail-fast behavior of the MBT tool caused test execution to terminate immediately upon encountering assertion failures, preventing further traversal of the model. The proposed retry and blacklist heuristics allowed execution to recover from transient failures and continue exploration after persistent localized failures. As demonstrated in the empirical results (Section 5.5), this behavior enabled MBT executions to continue beyond failure points that would otherwise terminate the test session, thereby allowing additional parts of the model to be exercised during a single execution run.

Several directions for future work emerge from this study. First, the proposed heuristics should be evaluated across a broader range of MBT tools and Systems Under Test (SUTs) to further assess their generality and applicability beyond the studied context. In particular, applying the extended GraphWalker implementation to additional industrial or open-source systems with different model structures and failure characteristics would provide stronger empirical evidence regarding the robustness and transferability of the proposed execution strategies. In particular, evaluating the heuristics across multiple MBT tools and systems under test would provide stronger evidence of their general applicability. Second, improved visualization of fault-tolerant MBT execution—such as highlighting retried and blacklisted nodes—could enhance result interpretation and support integration with existing test reporting and analytics tools. Finally, exploring additional execution-control policies and adaptive fault-tolerance strategies may further improve the effectiveness of online MBT in complex industrial environments. Also, a broader empirical investigation across different system types and failure modes is needed to assess whether higher retry counts yield improved recovery rates in other contexts. Evaluating adaptive or context-aware retry policies represents an important direction for future research.

CRediT authorship contribution statement

Vəhid Gəruslu: Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Alper Buğra Keleş:** Investigation, Formal analysis, Data

curation. **Yunus Balaman:** Resources, Methodology, Investigation. **Glen O'Donovan:** Writing – original draft, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have **NO known** competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Earlier support for this ongoing industry-academia collaborative project were provided by a TÜBİTAK-1509 grant from the Scientific and Technological Research Council of Turkey and a European ITEA4 R&D project named "TESTOMAT – The Next Level of Test Automation" (testomatproject.eu and itea4.org/project/testomatproject.html).

Appendix A. Source Code of the New Classes added to the MBT Tool to Implement the Two Heuristics

TryAgainStrategy.java

```
package org.graphwalker.core.machine;
import org.graphwalker.core.model.Edge;
import org.graphwalker.core.model.Element;
import org.graphwalker.core.model.Requirement;
import org.graphwalker.core.model.Vertex;
import java.util.List;
  import static org.graphwalker.core.model.Edge.RuntimeEdge;
/**
  * TryAgainStrategy
  * The TryAgainStrategy handles the way GraphWalker handles failures during test execution.
  *
  * The default way of handling a failure when executing a test, is to stop the test run and bail out.
  * This class goes back to the previous node and then re-executes the connecting edge between the previous and failed
  node
*/
public class TryAgainStrategy implements ExceptionStrategy {
    @Override
    public void handle(machine, MachineException exception) {
        context = exception.getContext();
        if(context.getCurrentElement() instanceof RuntimeEdge)
            if(((RuntimeEdge) context.getCurrentElement()).getTargetVertex().getNodeStatus().equals(NodeStatus.
  FAILED)){
                context.setExecutionStatus(ExecutionStatus.EXECUTING);
        else {
            //Has this node failed previously? if so stop the execution of the test
            if (((Vertex.RuntimeVertex)
context.getCurrentElement()).getNodeStatus().equals(NodeStatus.FAILED)) {
                context.setExecutionStatus(ExecutionStatus.FAILED);
              throw exception;
            }
            ((Vertex.RuntimeVertex) context.getCurrentElement()).setNodeStatus(NodeStatus.FAILED);
            Edge.RuntimeEdge nextEdge = (RuntimeEdge) context.getLastElement();
            //The source vertex of the edge would be the previous node from the failed node
            Vertex.RuntimeVertex vertex = nextEdge.getSourceVertex();
            //Set execution status to Executing
            //Set current element to the previous node
            //Set next element to be the edge connecting the previous node and the failed node
            context.setExecutionStatus(ExecutionStatus.EXECUTING);
            machine.getCurrentContext().setCurrentElement(vertex);
            machine.getCurrentContext().setNextEdgeTryAgain(nextEdge);
        }
    } // end of method
} //end of class
```

BlackListStrategy.java

```
package org.graphwalker.core.machine;
import org.graphwalker.core.model.Vertex;
import java.util.*;
import static org.graphwalker.core.model.Edge.RuntimeEdge;
/**
  *BlackListStrategy
  *The BlackListStrategy handles the way GraphWalker handles failures during allNodesThatAreNotCovered execution.
  *
  *The default way of handling a failure when executing a allNodesThatAreNotCovered, is to stop the
  *allNodesThatAreNotCovered run and bail out.
```

(*continued on next page*)

(*continued*)

```
   *This class goes back to the previous node and then continues the execution while avoiding the failed node/s
 */
public class BlackListStrategy implements ExceptionStrategy {
    @Override
    public void handle(machine, MachineException exception) {
        context = exception.getContext();
        List<Vertex.RuntimeVertex> allNodes = context.getModel().getVertices();
        List<RuntimeEdge> allEdges = context.getModel().getEdges();
        if (context.getCurrentElement() instanceof Vertex.RuntimeVertex) {
            ((Vertex.RuntimeVertex) context.getCurrentElement()).setNodeStatus(NodeStatus.FAILED);
            //a mapping from a node to its targets and sources
            HashMap<Vertex.RuntimeVertex, HashSet<Vertex.RuntimeVertex>> GTargetMap = new
  HashMap<Vertex.RuntimeVertex, HashSet<Vertex.RuntimeVertex>>();
            HashMap<Vertex.RuntimeVertex, HashSet<Vertex.RuntimeVertex>> GSourceMap = new
  HashMap<Vertex.RuntimeVertex, HashSet<Vertex.RuntimeVertex>>();
            //2 hashmaps in order to keep track of where we have been in the dfs and mem is used to hold the
  reachability of a node
            HashMap<Vertex.RuntimeVertex, Boolean> isVisited = new HashMap<Vertex.RuntimeVertex, Boolean>
  ();
            HashMap<Vertex.RuntimeVertex, Boolean> mem = new HashMap<Vertex.RuntimeVertex, Boolean>();
            for (Vertex.RuntimeVertex vertex: allNodes) {
                GTargetMap.put(vertex, new HashSet<>());
                GSourceMap.put(vertex, new HashSet<>());
                isVisited.put(vertex, false);
                for (RuntimeEdge edge: allEdges) {
                    if (edge.getSourceVertex().equals(vertex))
                        GTargetMap.get(vertex).add(edge.getTargetVertex());
                    if (edge.getTargetVertex().equals(vertex))
                        GSourceMap.get(vertex).add(edge.getSourceVertex());
                }
            }
            for (Vertex.RuntimeVertex vertex: allNodes)
              if (!dfs(vertex, GTargetMap, GSourceMap, isVisited, mem))
                    if(vertex.getNodeStatus().equals(NodeStatus.NOT_COVERED))
                      vertex.setNodeStatus(NodeStatus.NOT_REACHABLE);
            //execution status change, to keep the program running
            context.setExecutionStatus(ExecutionStatus.EXECUTING);
            //Set the current and next element to the appropriate nodes and edges depending on the surrounding edges
  of the failed node
            Vertex.RuntimeVertex newStart = ((RuntimeEdge) context.getLastElement()).getSourceVertex();
            boolean nextElementsSet = false;
            for (int i = 0; i < allEdges.size(); i++) {
                if(allEdges.get(i).getSourceVertex().equals(context.getCurrentElement()) && allEdges.get(i).
  getTargetVertex().equals(((RuntimeEdge)context.getLastElement()).getSourceVertex())){
                    machine.getCurrentContext().setCurrentElement(newStart);
                    machine.getCurrentContext().setNextEdgeTryAgain(allEdges.get(i));
                    nextElementsSet = true;
              }
                if(nextElementsSet)
                    break;
            }
            if(!nextElementsSet)
                machine.getCurrentContext().setCurrentElement(newStart);
        }
    }// end of method
    /**
    * returns Booleans based off if the node in question is reachable or not
    */
    public Boolean dfs(Vertex.RuntimeVertex root,
                                    HashMap<Vertex.RuntimeVertex, HashSet<Vertex.RuntimeVertex>>
  GTargetMap,
                                    HashMap<Vertex.RuntimeVertex, HashSet<Vertex.RuntimeVertex>>
  GSourceMap,
                                    HashMap<Vertex.RuntimeVertex, Boolean> isVisited,
                                    HashMap<Vertex.RuntimeVertex, Boolean> mem)
    {
        if(isVisited.get(root))
            return mem.get(root);
        if(root.getNodeStatus().equals(NodeStatus.FAILED)){
            mem.put(root, false);
            return false;
        }
        //mark root as visited
        isVisited.put(root, true);
        Boolean isAnySourceReachable = false;
        //logic for marking root as reachable
        for(Vertex.RuntimeVertex source: GSourceMap.get(root)) {
```

(*continued on next page*)

(*continued*)

```
            isAnySourceReachable = source.getNodeStatus().equals(NodeStatus.COVERED) || isAnySourceReachable;
            if (source.getNodeStatus().equals(NodeStatus.NOT_COVERED))
                if (dfs(source, GTargetMap, GSourceMap, isVisited, mem))
                    isAnySourceReachable = true;
        }
        mem.put(root, isAnySourceReachable);
        for(Vertex.RuntimeVertex target: GTargetMap.get(root))
            dfs(target, GTargetMap, GSourceMap, isVisited, mem);
        return isAnySourceReachable;
    }// end of method
} //end of class
```

Data availability

URL of data is in the paper

References

[1] M. Utting, B. Legeard, Practical Model-Based testing: a Tools Approach, Elsevier, 2010.

[2] A. Kramer, B. Legeard, Model-Based Testing Essentials-Guide to the ISTQB Certified Model-Based Tester, John Wiley & Sons, 2016.

[3] W. Elmendorf, "Automated design of program test libraries," IBM Technical report, TR 00.2089, https://benderrbt.com/Automated%20Design%20of%20Program%20Test%20Libraries%20-%201970.pdf, 1970.

[4] A.D. Neto, R. Subramanyan, M. Vieira, G.H. Travassos, F. Shull, Improving evidence about software technologies: a look at model-based testing, IEEE Soft. 25 (3) (2008) 10–13.

[5] S.R. Dalal et al., "Model-based testing in practice," in Proceedings of International Conference on Software Engineering, 1999, pp. 285–294.

[6] R.V. Binder, B. Legeard, A. Kramer, Model-based testing: where does it stand? Commun. ACM 58 (2) (2015) 52–56.

[7] I. Schieferdecker, Model-based testing, IEEE Softw. 29 (1) (2012) 14.

[8] P.C. Jorgensen, The Craft of Model-Based testing, Auerbach Publications, 2017.

[9] B. Elodie, A. Fabrice, L. Bruno, B. Arnaud, Lightweight model-based testing for enterprise IT, in: IEEE International Conference on Software Testing, Verification and Validation, IEEE, 2018, pp. 224–230.

[10] J. Shore, Fail fast, IEEE Soft. 21 (5) (2004) 21–25.

[11] K. Karl, "ExceptionStrategy of the GraphWalker code-base," https://github.com/GraphWalker/graphwalker-project/blob/master/graphwalker-core/src/main/java/org/graphwalker/core/machine/ExceptionStrategy.java, Last accessed: June 2025.

[12] K. Karl, "Online discussions about the default fail strategy of Graphwalkers (fail fast)," https://groups.google.com/g/graphwalker/c/p7EA82LJu5Y?pli=1, Last accessed: June 2025.

[13] O. Parry, G.M. Kapfhammer, M. Hilton, P. McMinn, A survey of flaky tests, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (1) (2021) 1–74.

[14] V. Garousi, A.B. Keleş, Y. Balaman, Z.Ö. Güler, A. Arcuri, Model-based testing in practice: an experience report from the web applications domain, J. Syst. Softw. 180 (2021) 111032.

[15] R. Natella and D. Cotroneo, "Emulation of transient software faults for dependability assessment: a case study," in European Dependable Computing Conference, 2010, pp. 23–32.

[16] V. Garousi, M. Felderer, and M.V. Mäntylä, "The need for multivocal literature reviews in software engineering: complementing systematic literature reviews with grey literature," in International Conference on Evaluation and Assessment in Software Engineering, Limmerick, Ireland, 2016, pp. 171–176.

[17] V. Garousi, M. Felderer, M.V. Mäntylä, Guidelines for including grey literature and conducting multivocal literature reviews in software engineering, Inf. Softw . Technol. 106 (2019) 101–121.

[18] V. Garousi, M. Felderer, M.V. Mäntylä, A. Rainer, Benefitting from the grey literature in software engineering research. Contemporary Empirical Methods in Software Engineering, Springer, 2020 editor: M. Felderer, G.H. Travassos.

[19] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in Proceedings of the international symposium on foundations of software engineering, 2014, pp. 643–653.

[20] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: automatically detecting flaky tests," in Proceedings of international conference on software engineering, 2018, pp. 433–444.

[21] Dilato Innovative Technology, "Fault tolerance and exception handling in test automation," www.dilatoit.com/2024/02/29/fault-tolerance-and-exception-handling-in-test-automation.html, Last accessed: Jan 2026.

[22] The Linux Foundation, "Flaky test handler," plugins.jenkins.io/flaky-test-handler/, Last accessed: Jan 2026.

[23] Atlassian, "Taming test flakiness: how we built a scalable tool to detect and manage flaky TestsFlaky test handler," www.atlassian.com/blog/atlassian-engineering/taming-test-flakiness-how-we-built-a-scalable-tool-to-detect-and-manage-flaky-tests , Last accessed: Jan 2026.

[24] A. Patel, "Handling flaky tests at scale: auto detection & suppression," slack.engineering/handling-flaky-tests-at-scale-auto-detection-suppression/, Last accessed: Jan 2026.

[25] Y. Koroglu, M. Beyazıt, O. Kilincceker, S. Demeyer, F. Wotawa, Towards improving automated testing with GraphWalker, in: IEEE International Conference on Software Testing, Verification and Validation Workshops, IEEE, 2025, pp. 54–58.

[26] J. Korhonen, Automated Model Generation Using Graphwalker Based On Given-When-Then Specifications, Mälardalen University, 2020. Bachelor Thesis.

[27] P. Akpinar, M.S. Aktas, A.B. Keles, Y. Balaman, Z.O. Guler, O. Kalipsiz, Web application testing with model based testing method: case study, in: International Conference on Electrical, Communication, and Computer Engineering, IEEE, 2020, pp. 1–6.

[28] V. Garousi, A.B. Keleş, Y. Balaman, and Z.Ö. Güler, "Set of demo and experience videos on model-based testing (MBT) innovations in Testinium A.Ş," www.youtube.com/playlist?list=PLsOt2CHZC7PqQ-fzwk_6JEibyIOqeJN0e, 2021.

[29] T. Ahmad, J. Iqbal, A. Ashraf, D. Truscan, I. Porres, Model-based testing using UML activity diagrams: a systematic mapping study, Comput. Sci. Rev. 33 (2019) 98–112, https://doi.org/10.1016/j.cosrev.2019.07.001.

[30] R. Marinescu, C. Seceleanu, H. Le Guen, P. Pettersson, Chapter three - A research overview of tool-supported model-based testing of requirements-based designs, in: A.R. Hurson (Ed.), Advances in Computers, Advances in Computers, 98, Elsevier, 2015, pp. 89–140.

[31] V. Garousi, A.B. Keleş, Y. Balaman, Z.Ö. Güler, Test automation using the Gauge framework: experience and best practices, in: International Conference on Computational Science and its Applications, 2020, pp. 458–470.

[32] V. Garousi, A.B. Keleş, Z.Ö. Güler, Y. Balaman, Executable natural-language test specifications: a test-automation experience report, in: Proceedings of the Turkish National Software Engineering Symposium (UYMS), 2019.

[33] V. Garousi, A.B. Keles, Application of TMMi to improve test approaches and processes: experience from two Turkish companies. An invited talk for the TMMi Hungarian Local Chapter, 2021, in: www.slideshare.net/vahidgarousi/application-of-tmmi-to-improve-test-approaches-and-processes-experience-from-two-turkish-companies.

[34] V.J. Garousi, Nithin; Jafarov, Zafar; Keleş, Alper Buğra; Değirmenci, Sevde; Özdemir, Ece; Zarringhalami, Ryan, "AI-powered test automation tools: a systematic review and empirical evaluation," arXiv pre-print 2025. [Online]. Available: www.arxiv.org/abs/2409.00411.

[35] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering: An Introduction, Kluwer Academic Publishers, 2000.